



Programación de Aplicaciones Gráficas

Sesión 1: Configurar un proyecto para usar OpenGL Core Profile

Introducción.....	2
¿Qué es GLFW?.....	2
¿Qué es GLAD?.....	2
Objetivos.....	3
Material.....	3
Paso 0: Instalación del IDE.....	3
Paso 1: Creación del proyecto.....	3
Añadir el archivo main.cpp.....	3
Instalación de bibliotecas en GNU/Linux.....	4
Instalación de bibliotecas en macOS.....	4
Instalación de bibliotecas en Windows para uso con CLion (gestor de paquetes vcpkg).....	4
Instalación de GLAD.....	4
Escribir el esqueleto base de una aplicación OpenGL Core Profile.....	7
Añadir las bibliotecas a enlazar durante la compilación.....	9
CLion en GNU/Linux.....	9
CLion en macOS.....	9
CLion y vcpkg en Windows.....	9
Compilar y ejecutar.....	10
Refinar el programa.....	10
Trabajo autónomo.....	12
Entrega.....	13
Material a preparar para la próxima sesión de prácticas.....	13
Anexo I: Pasos para la integración de vcpkg en un proyecto CLion.....	14
Introducción.....	14
Primer paso: proyecto de partida.....	14
Segundo paso: integración de vcpkg en el proyecto.....	15

Tercer paso: añadir bibliotecas al proyecto.....	16
Cuarto paso: solución de problemas.....	20
Cambiar o añadir una instancia de vcpkg.....	22

Introducción

En esta asignatura vamos a usar OpenGL en modo Core Profile. Para ello nos apoyaremos en varias bibliotecas: GLFW, GLAD, ImGUI, GLM, Assimp y LodePNG, aunque en esta primera sesión sólo vamos a necesitar las dos primeras.

En este documento vamos a ver cómo configurar un proyecto para usar estas bibliotecas. Sin embargo, no vamos a centrarnos en un IDE concreto (aunque pongamos ejemplos utilizando alguno). Dependiendo del IDE que quieras utilizar (CLion, Visual Studio, ...) deberás hacer las adaptaciones correspondientes.

¿Qué es GLFW?

[GLFW](#) (OpenGL FrameWork) es una biblioteca de C++ que proporciona una API para crear ventanas con un contexto gráfico OpenGL Core Profile asociado, capaz de recibir eventos de teclado, ratón, etc.

GLFW permite definir un contexto gráfico OpenGL con las características que necesitemos, tales como si se usa *antialiasing* o no, el número de muestras para *antialiasing*, el número de bits para codificar el color o la profundidad (Z-buffer), la versión de OpenGL que necesitamos, etc.

También permite registrar funciones (*callbacks*) que se llamarán cuando se produzcan ciertos eventos en nuestra ventana OpenGL, como que haya que redibujarla, que se haya redimensionado, o que se haya pulsado una tecla o un botón del ratón.

¿Qué es GLAD?

[GLAD](#) es una biblioteca de C++ que proporciona acceso eficiente a las funciones de OpenGL Core Profile en tiempo de ejecución, lo que se denomina una **biblioteca de carga de OpenGL**.

La mayoría de los compiladores proporcionan por defecto acceso a cabeceras (.h) OpenGL que permiten acceder a la funcionalidad de OpenGL 1.1. Eso significa que aunque en nuestro equipo esté instalada una versión posterior de OpenGL (en estas prácticas usaremos la 4.3), no tendremos acceso a ella desde nuestros programas de prácticas, a menos que utilicemos una biblioteca de carga de OpenGL.

GLAD es una biblioteca que tiene dos objetivos principales:

- Proporcionar acceso a las cabeceras que incluyen las nuevas funciones de OpenGL Core Profile. Así nuestro código podrá compilarse. Por eso, **el include de GLAD siempre debe estar antes que el de GLFW**.

- Enlazar en tiempo de ejecución con las bibliotecas adecuadas, de forma que se ejecuten las nuevas funcionalidades de OpenGL Core Profile. Para ello, GLAD comprueba en tiempo de ejecución la versión del contexto gráfico OpenGL que se está usando, y carga las bibliotecas adecuadas para esa versión. Esta es la razón por la que **siempre debe inicializarse GLFW antes que GLAD en nuestro programa principal**.

En [este enlace](#) tienes más información acerca del sentido y funcionamiento de GLAD y otras bibliotecas similares.

Objetivos

En esta sesión aprenderemos a crear y configurar un proyecto en C++ que permita abrir una ventana OpenGL en modo Core Profile, capaz de recibir y procesar eventos de usuario tales como pulsaciones de teclas o de los botones del ratón.

Material

Necesitarás tener instalado un IDE de programación en C++. Te recomendamos que utilices [CLion](#) (si te creas una cuenta de estudiante, es gratuito), aunque Visual Studio, NetBeans, Eclipse o cualquier otro deberían servirte, cada uno con sus peculiaridades para añadir bibliotecas y compilar el código enlazándolas.

Paso 0: Instalación del IDE

Si no lo tuviéramos instalado aún, deberemos instalar el entorno de desarrollo (IDE) de nuestra elección. En las páginas web de cada IDE suele haber una guía de instalación para los diferentes sistemas operativos. En el caso de CLion, la guía de instalación está disponible en el siguiente enlace: <https://www.jetbrains.com/help/clion/installation-guide.html>

Paso 1: Creación del proyecto

Utilizando el IDE que hayas elegido, crea un proyecto C/C++ vacío basado en CMake. Sobre este proyecto iremos añadiendo las características que necesitamos.

Añadir el archivo main.cpp

La mayoría de IDEs añaden por defecto un archivo main.cpp vacío al nuevo proyecto, pero si no fuera así, añádelo con las opciones que te dé el entorno de programación.

Ahora hay que escribir el texto de ese archivo. Comenzaremos por una versión básica, aunque más adelante la cambiaremos:

```
#include <iostream>
```

```
#include <cstdlib>

int main()
{
    std::cout << "Starting application PAG " << std::endl;
}
```

Prueba a compilar el código, para asegurarte de que por el momento, todo funciona correctamente.

Instalación de bibliotecas en GNU/Linux

En el caso de sistemas GNU/Linux, el gestor de paquetes del sistema operativo nos permite instalar fácilmente las bibliotecas que necesitamos. Si optamos por una distribución basada en Ubuntu/Debian, podemos instalar los paquetes ejecutando la siguiente instrucción por consola:

```
apt-get install build-essential libglfw3-dev libglm-dev libassimp-dev
```

Además, si optamos por instalar CLion como entorno de desarrollo, necesitaremos instalar el paquete *libfuse2*.

Para añadir las bibliotecas a nuestro proyecto, necesitaremos incluir en el archivo de configuración de CMake (CMakeLists.txt) las instrucciones correspondientes. Más adelante veremos cómo hacerlo.

Instalación de bibliotecas en macOS

En sistemas macOS, lo más cómodo es utilizar algún gestor de paquetes similar a los de los sistemas GNU/Linux. Quizá el más popular hoy en día sea homebrew (<https://brew.sh>). Desde su página web se puede consultar cómo hacer la instalación del gestor y cómo utilizarlo para instalar las bibliotecas.

Instalación de bibliotecas en Windows para uso con CLion (gestor de paquetes vcpkg)

En el caso de Windows, podemos utilizar el gestor de paquetes vcpkg para instalar automáticamente las bibliotecas y sus dependencias.

En el [Anexo I](#) tienes información detallada para la integración de vcpkg en un proyecto CLion.

Instalación de GLAD

La mejor opción para instalar GLAD es hacerlo manualmente. Esto nos permite ajustar la configuración de la biblioteca a nuestras necesidades.

Para instalar GLAD manualmente, debemos acceder a la página web de esta biblioteca (<https://glad.dav1d.de>) y configurar la versión de GLAD que queremos instalar. Debemos elegir el

lenguaje C++, el perfil “core” de la especificación OpenGL y al menos la versión 4.3. No es necesario añadir las extensiones para esta asignatura. La Figura 1 muestra esta configuración.

Glad

Multi-Language GL/GLES/EGL/GLX/WGL Loader-Generator based on the official specs.

Language

C/C++

Specification

OpenGL

API

gl

Version 4.3

gles1

None

gles2

None

glsc2

None

Profile

Core

Extensions

Search

GL_3DFX_multisample
GL_3DFX_tbuffer
GL_3DFX_texture_compression_FXT1
GL_AMD_blend_minmax_factor
GL_AMD_conservative_depth
GL_AMD_debug_output
GL_AMD_depth_clamp_separate
GL_AMD_draw_buffers_blend

Search

ADD LIST

ADD ALL

REMOVE ALL

Options

☒ Generate a loader

☐ Omit KHR (due to recent changes to the specification, this may not work anymore)

☐ Local Files

GENERATE

Figura 1

Al pulsar el botón “Generate”, se nos da la opción de descargar la versión de GLAD que hemos configurado (Figura 2). Basta con descargar el archivo en formato ZIP, que contiene las dos carpetas superiores.

Glad

Generated files. These files are not permanent!

Name ^	Last modified	Size
 include	2026-08-27 09:02:24	-
 src	2026-08-27 09:02:24	-
 glad.zip	2026-08-27 09:02:24	387.9 kB

Permalink:

<https://glad.dav1d.de/#language=c&specification=gl&api=gl%3D4.3&api=gles1%3Dnone&api=gles2%3Dnone&api=glsc2%3D>

Figura 2

Podemos instalar GLAD globalmente en el sistema utilizando la información descargada, pero lo más cómodo es incluir GLAD únicamente en nuestro proyecto, creando, por ejemplo, una carpeta GLAD, y descomprimiendo ahí el archivo ZIP (Figura 3). Tendremos que añadir a la configuración de CMake las líneas correspondientes para compilar GLAD dentro de nuestro proyecto.

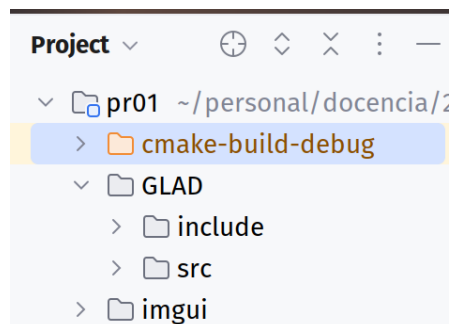


Figura 3

En el archivo CMakeLists.txt, podemos añadir instrucciones para la compilación de GLAD de la siguiente forma:

```
# Cabecera del CMakeLists.txt ...

# Define variables agrupando nombres de archivo
file( GLOB MY_FILES *.cpp )    # Código fuente de tu proyecto
file( GLOB GLAD_FILES GLAD/src/*.c )    # Código fuente de GLAD

# Indica qué archivos hay que compilar para generar el ejecutable
add_executable( ${PROJECT_NAME} ${MY_FILES}
               ${GLAD_FILES} )

# Indica directorios (adicionales a los por defecto) para buscar includes
target_include_directories( ${PROJECT_NAME} PUBLIC GLAD/include )
```

```
# Resto del CMakeLists.txt ...
```

NOTA: En los siguientes ejemplos de código, vamos a partir del supuesto de que GLAD se ha instalado de esta forma.

Escribir el esqueleto base de una aplicación OpenGL Core Profile

Es el momento de completar el código del archivo `main.cpp`. Para ello, copia el siguiente listado en tu archivo `main.cpp`. El código está comentado, pero no dudes en preguntar cualquier duda que te surja.

```
#include <iostream>
// IMPORTANTE: El include de GLAD debe estar siempre ANTES de el de GLFW
#include <glad/glad.h>
#include <GLFW/glfw3.h>

int main()
{
    std::cout << "Starting Application PAG - Prueba 01" << std::endl;

    // - Inicializa GLFW. Es un proceso que sólo debe realizarse una vez en la aplicación
    if ( glfwInit () != GLFW_TRUE )
    {
        std::cout << "Failed to initialize GLFW" << std::endl;
        return -1;
    }

    // - Definimos las características que queremos que tenga el contexto gráfico
    // OpenGL de la ventana que vamos a crear. Por ejemplo, el número de muestras o el
    // modo Core Profile.
    glfwWindowHint ( GLFW_SAMPLES, 4 ); // - Activa antialiasing con 4 muestras.
    glfwWindowHint ( GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE ); // - Esta y las 2
    glfwWindowHint ( GLFW_CONTEXT_VERSION_MAJOR, 4 ); // siguientes activan un contexto
    glfwWindowHint ( GLFW_CONTEXT_VERSION_MINOR, 3 ); // OpenGL Core Profile 4.3.

    // - Definimos el puntero para guardar la dirección de la ventana de la aplicación y
    // la creamos
    GLFWwindow *window;
    // - Tamaño, título de la ventana, en ventana y no en pantalla completa,
    // sin compartir recursos con otras ventanas.
    window = glfwCreateWindow ( 1024, 576, "PAG Introduction", nullptr, nullptr );

    // - Comprobamos si la creación de la ventana ha tenido éxito.
    if ( window == nullptr )
    {
        std::cout << "Failed to open GLFW window" << std::endl;
        glfwTerminate (); // - Liberamos los recursos que ocupaba GLFW.
        return -2;
    }
}
```

```

// - Hace que el contexto OpenGL asociado a la ventana que acabamos de crear pase a
// ser el contexto actual de OpenGL para las siguientes llamadas a la biblioteca
glfwMakeContextCurrent ( window );

// - Ahora inicializamos GLAD.
if ( !gladLoadGLLoader ( (GLADloadproc) glfwGetProcAddress ) )
{
    std::cout << "GLAD initialization failed" << std::endl;
    glfwDestroyWindow ( window ); // - Liberamos los recursos que ocupaba GLFW.
    window = nullptr;
    glfwTerminate ();
    return -3;
}

// - Interrogamos a OpenGL para que nos informe de las propiedades del contexto
// 3D construido.
std::cout << glGetString ( GL_RENDERER ) << std::endl
          << glGetString ( GL_VENDOR ) << std::endl
          << glGetString ( GL_VERSION ) << std::endl
          << glGetString ( GL_SHADING_LANGUAGE_VERSION ) << std::endl;

// - Establecemos un gris medio como color con el que se borrará el frame buffer.
// No tiene por qué ejecutarse en cada paso por el ciclo de eventos.
glClearColor ( 0.6, 0.6, 0.6, 1.0 );

// - Le decimos a OpenGL que tenga en cuenta la profundidad a la hora de dibujar.
// No tiene por qué ejecutarse en cada paso por el ciclo de eventos.
glEnable ( GL_DEPTH_TEST );

// - Ciclo de eventos de la aplicación. La condición de parada es que la
// ventana principal deba cerrarse. Por ejemplo, si el usuario pulsa el
// botón de cerrar la ventana (la X).
while ( !glfwWindowShouldClose ( window ) )
{
    // - Borra los buffers (color y profundidad)
    glClear ( GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT );

    // - GLFW usa un doble buffer para que no haya parpadeo. Esta orden
    // intercambia el buffer back (en el que se ha estado dibujando) por el
    // que se mostraba hasta ahora (front).
    glfwSwapBuffers ( window );
    // - Obtiene y organiza los eventos pendientes, tales como pulsaciones de
    // teclas o de ratón, etc. Siempre al final de cada iteración del ciclo
    // de eventos y después de glfwSwapBuffers(window);
    glfwPollEvents ();
}

// - Una vez terminado el ciclo de eventos, liberar recursos, etc.
std::cout << "Finishing application pag prueba" << std::endl;

glfwDestroyWindow ( window ); // - Cerramos y destruimos la ventana de la aplicación.
window = nullptr;
glfwTerminate (); // - Liberamos los recursos que ocupaba GLFW.
}

```


Añadir las bibliotecas a enlazar durante la compilación

Para poder compilar el código que hemos preparado, necesitamos añadir a nuestro proyecto las bibliotecas OpenGL, GLAD y GLFW (más adelante añadiremos otras). Vamos a resumir brevemente este proceso.

CLion en GNU/Linux

En GNU/Linux sólo necesitamos modificar el archivo de configuración CMakeLists.txt del proyecto para indicar qué bibliotecas hay que enlazar, mientras que para compilar GLAD dentro de nuestra aplicación (asumiendo que está en una carpeta dentro del proyecto, tal y como se indicó antes), vamos a incluir sus archivos de código fuente en la lista de archivos a compilar. Con estos cambios, el archivo CMakeLists.txt queda como sigue:

```
cmake_minimum_required(VERSION 3.27)
project(pr01)

set(CMAKE_CXX_STANDARD 17)

# Define variables agrupando nombres de archivo
file( GLOB MY_FILES *.cpp )
file( GLOB GLAD_FILES GLAD/src/*.c )

# Indica qué archivos hay que compilar para generar el ejecutable
add_executable( ${PROJECT_NAME} ${MY_FILES}
               ${GLAD_FILES} )

# Indica directorios (adicionales a los por defecto) para buscar includes
target_include_directories( ${PROJECT_NAME} PUBLIC GLAD/include )

# Indica bibliotecas a enlazar en el ejecutable
target_link_libraries( ${PROJECT_NAME} GL glfw )
```

CLion en macOS

Utilizando homebrew, la instalación de bibliotecas en macOS es similar a los sistemas GNU/Linux. Las modificaciones en el archivo CMakeLists.txt también son similares.

CLion y vcpkg en Windows

Para añadir las bibliotecas en tu proyecto CLion en Windows, tienes que seguir los pasos indicados en el [Anexo I](#) de este documento.

Compilar y ejecutar

Si hemos creado el proyecto y añadido las bibliotecas correctamente, deberíamos poder compilar el código sin problema. Al ejecutarlo, veremos la ventana OpenGL que nos proporciona GLFW (Figura 4). Por ahora el programa gráfico es muy simple ya que sólo borra el *framebuffer* con un color gris.

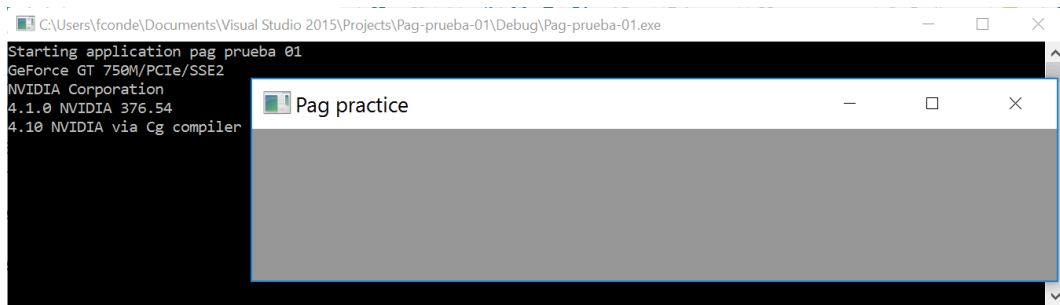


Figura 4

Mientras el programa gráfico está en ejecución, podemos ver la consola que lo lanza y los mensajes que se vayan imprimiendo. Para cerrar el programa, como con cualquier otro, basta con pulsar la "X" de la ventana.

Refinar el programa

Para terminar por ahora, vamos a añadir algunos *callbacks* a nuestra aplicación. De esta forma la prepararemos para recibir eventos y poder responder a ellos.

Edita el programa `main.cpp` para que quede de la siguiente forma (ten en cuenta que el siguiente listado no está completo):

```
...
#include <GLFW\glfw3.h>

// - Esta función callback será llamada cuando GLFW produzca algún error
void error_callback ( int errno, const char* desc )
{
    std::string aux (desc);
    std::cout << "Error de GLFW número " << errno << ": " << aux << std::endl;
}

// - Esta función callback será llamada cada vez que el área de dibujo
// OpenGL deba ser redibujada.
void window_refresh_callback ( GLFWwindow *window )
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    // - GLFW usa un doble buffer para que no haya parpadeo. Esta orden
    // intercambia el buffer back (que se ha estado dibujando) por el
    // que se mostraba hasta ahora front. Debe ser la última orden de
    // este callback
    glfwSwapBuffers ( window );
    std::cout << "Refresh callback called" << std::endl;
}
```

```

}

// - Esta función callback será llamada cada vez que se cambie el tamaño
// del área de dibujo OpenGL.
void framebuffer_size_callback ( GLFWwindow *window, int width, int height )
{ glViewport ( 0, 0, width, height );
  std::cout << "Resize callback called" << std::endl;
}

// - Esta función callback será llamada cada vez que se pulse una tecla
// dirigida al área de dibujo OpenGL.
void key_callback ( GLFWwindow *window, int key, int scancode, int action, int mods )
{ if ( key == GLFW_KEY_ESCAPE && action == GLFW_PRESS )
  { glfwSetWindowShouldClose(window, GLFW_TRUE);
  }
  std::cout << "Key callback called" << std::endl;
}

// - Esta función callback será llamada cada vez que se pulse algún botón
// del ratón sobre el área de dibujo OpenGL.
void mouse_button_callback ( GLFWwindow *window, int button, int action, int mods )
{ if ( action == GLFW_PRESS )
  { std::cout << "Pulsado el botón: " << button << std::endl;
  }
  else if ( action == GLFW_RELEASE )
  { std::cout << "Soltado el botón: " << button << std::endl;
  }
}

// - Esta función callback será llamada cada vez que se mueva la rueda
// del ratón sobre el área de dibujo OpenGL.
void scroll_callback ( GLFWwindow *window, double xoffset, double yoffset )
{ std::cout << "Movida la rueda del ratón " << xoffset
  << " Unidades en horizontal y " << yoffset
  << " unidades en vertical" << std::endl;
}

int main()
{ std::cout << "Starting Application PAG - Prueba 01" << std::endl;

  // - Este callback hay que registrarlo ANTES de llamar a glfwInit
  glfwSetErrorCallback ( (GLFWerrorfun) error_callback );

  ...
  std::cout << glGetString ( GL_SHADING_LANGUAGE_VERSION ) << std::endl;

  // - Registramos los callbacks que responderán a los eventos principales
  glfwSetWindowRefreshCallback ( window, window_refresh_callback );
  glfwSetFramebufferSizeCallback ( window, framebuffer_size_callback );
  glfwSetKeyCallback ( window, key_callback );
  glfwSetMouseButtonCallback ( window, mouse_button_callback );
  glfwSetScrollCallback ( window, scroll_callback );
}

```

```

// - Establecemos un gris medio como color con el que se borrará el
// frame buffer.
// No tiene por qué ejecutarse en cada paso por el ciclo de eventos.
glClearColor ( 0.6, 0.6, 0.6, 1.0 );

// - Le decimos a OpenGL que tenga en cuenta la profundidad a la hora de
// dibujar.
// No tiene por qué ejecutarse en cada paso por el ciclo de eventos.
glEnable ( GL_DEPTH_TEST );

// - Ciclo de eventos de la aplicación. La condición de parada es que la
// ventana principal deba cerrarse, por ejemplo, si el usuario pulsa el
// botón de cerrar la ventana (la X).
while ( !glfwWindowShouldClose ( window ) )
{ // - Obtiene y organiza los eventos pendientes, tales como pulsaciones
  // de teclas o de ratón, etc. Siempre al final de cada iteración del
  // ciclo de eventos y después de glfwSwapBuffers ( window );
  glfwPollEvents ();
}
...
}

```

Las funciones `error_callback`, `window_refresh_callback`, `framebuffer_size_callback`, `key_callback`, `mouse_button_callback`, y `scroll_callback` son las funciones que se llamarán cada vez que el evento correspondiente se dé en nuestra aplicación. Son funciones normales de C, no son métodos.

Como puedes ver, la llamada a OpenGL que borra el frame buffer `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)`; que en la versión anterior del programa se ejecutaba en cada paso por el ciclo de eventos, se ha movido al *callback* `window_refresh_callback`, de forma que sólo se ejecuta cuando la ventana debe redibujarse.

Trabajo autónomo

Un código bien desacoplado en el que las distintas tareas de la aplicación estén distribuidas en distintas clases es fundamental. En esta asignatura se valorará especialmente que tu código esté bien hecho.

- **Implementa** un *callback* para la aplicación básica que hemos presentado, que según el desplazamiento de la rueda del ratón, cambie el color de fondo de la ventana. Ten en cuenta, tal y como hemos visto en el tema de conceptos básicos, que los valores de color deben pertenecer al rango $[0,1]$. Por tanto, cuando calcules el nuevo valor para cada una de las componentes de color (R, G, B, A), deberás asegurarte de que el resultado no sobrepasa esos límites.

Puede servirte de ayuda consultar la documentación de GLFW relativa a la gestión de los eventos de usuario (https://www.glfw.org/docs/latest/input_guide.html#scrolling).

- Como segunda parte de esta primera práctica, un **ejercicio de reflexión** (¡OJO! no tienes que implementar nada) sobre estos temas:
 - o Una buena práctica que puedes aplicar es crear un espacio de nombres¹ (llamado “PAG”, por ejemplo) para tus clases. Así te aseguras no tener conflictos con otros nombres de clases o tipos de las bibliotecas que utilices.
 - o En nuestra aplicación debería haber una clase `PAG::Renderer` que se encargue de encapsular todo lo relativo al dibujado de la escena 3D en el contexto gráfico OpenGL. Esa clase debería tener un método llamado `refrescarVentana()` que debería llamarse cada vez que la ventana deba redibujarse. El problema es que no podemos registrar como *callback* para el evento `window_refresh` al método `PAG::Renderer::refrescarVentana()`, ya que no podemos registrar como *callbacks* métodos de clases, sino funciones sencillas de C.

¿Cómo solucionar este problema? ¿Cómo podemos llamar desde nuestras funciones *callback* en C a los métodos de la clase `PAG::Renderer` que responden a los distintos eventos? Todo ello habría de hacerse de la forma más encapsulada (es decir menos acoplada) posible.

Pistas para plantear la solución:

- ¿Cómo debería declararse el objeto de la clase `PAG::Renderer` que actúa como renderer de la aplicación?
- ¿Qué módulo de la aplicación debería inicializarlo?

Entrega

1. Sube tu proyecto a un servicio git online (Codeberg, GitHub, GitLab o similar).
2. Incluye en el `README.md` del proyecto la descripción de la solución que propones al ejercicio de reflexión planteado (sugerencia: incluye no sólo la explicación en texto, sino también un diagrama de clases UML). Esta solución NO SE IMPLEMENTA EN EL CÓDIGO.
3. Configura tu proyecto de tal forma que se pueda acceder de forma anónima al mismo.
4. A través del espacio de la asignatura en la plataforma PLATEA, en la tarea titulada **Práctica 1** que se encuentra en el espacio de prácticas de la asignatura, deberás indicar la URL de acceso a tu proyecto.

Material a preparar para la próxima sesión de prácticas

Para la próxima sesión de prácticas cada alumno deberá traer al aula un proyecto C++ para usar GLFW y GLAD igual que el que hemos construido en esta sesión. Eso incluye la gestión de los eventos.

¹ <https://www.geeksforgeeks.org/cpp/namespace-in-c>

Anexo I: Pasos para la integración de vcpkg en un proyecto CLion

Introducción

Vamos a aprender a configurar un proyecto en CLion que haga uso del gestor de paquetes vcpkg para solucionar las dependencias en el código.

vcpkg² es una herramienta gratuita desarrollada por Microsoft para instalar y configurar automáticamente las bibliotecas que necesitemos para desarrollar nuestras aplicaciones en C/C++. Está mantenido por el equipo de C++ de Microsoft y por personas externas del mundo del código abierto. CLion instala por defecto un plugin para vcpkg, por lo que salvo que lo hayamos desinstalado manualmente, debería estar disponible.

Primer paso: proyecto de partida

El primer paso es abrir el proyecto de partida en CLion. Podemos trabajar con un proyecto ya existente, o bien crear un proyecto nuevo en blanco (es lo que vamos a hacer nosotros). Una vez hecho esto, vamos a abrir el panel de vcpkg. Si no estuviera el icono disponible en las barras laterales de CLion, lo podemos encontrar haciendo clic en los tres puntos de la parte superior izquierda de la ventana (Figura A1.1).

Una vez activado el plugin, se abrirá un panel en la parte inferior de la ventana de CLion, desde donde podremos trabajar con vcpkg (Figura A1.2).

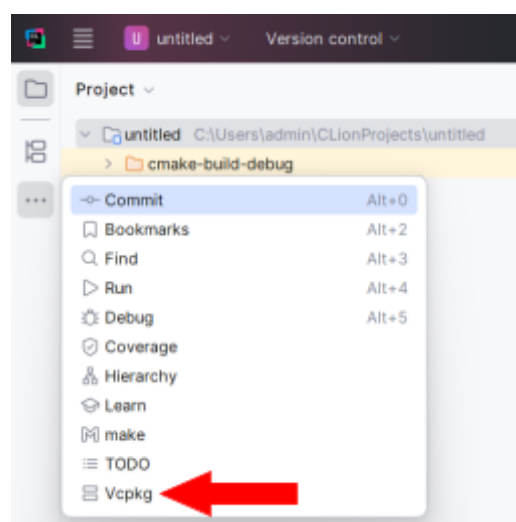


Figura A1.1

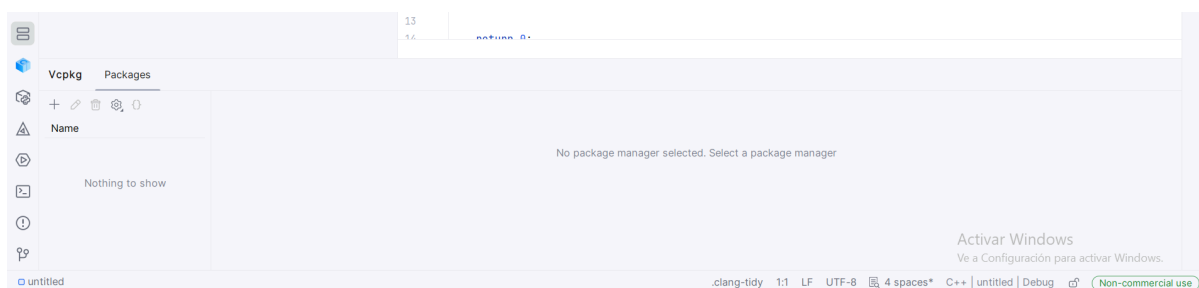


Figura A1.2

² <https://vcpkg.io>

Segundo paso: integración de vcpkg en el proyecto

Si ya hubiéramos integrado vcpkg en otro proyecto, nos aparecería disponible en el panel de la izquierda de la Figura A1.2. Sin embargo, en este caso podemos ver que vcpkg no ha sido utilizado anteriormente, por lo que en primer lugar, tendremos que descargar la base del sistema (crear una instancia de vcpkg en nuestro equipo³). Para hacer esto, haremos clic en el botón con el signo “+” a la izquierda del panel (Figura A1.3).

Una vez hecho esto, se abrirá una ventana en la que CLion nos permite configurar el directorio de instalación del sistema vcpkg, así como integrarlo en el proceso de construcción de nuestro ejecutable (Figura A1.4).

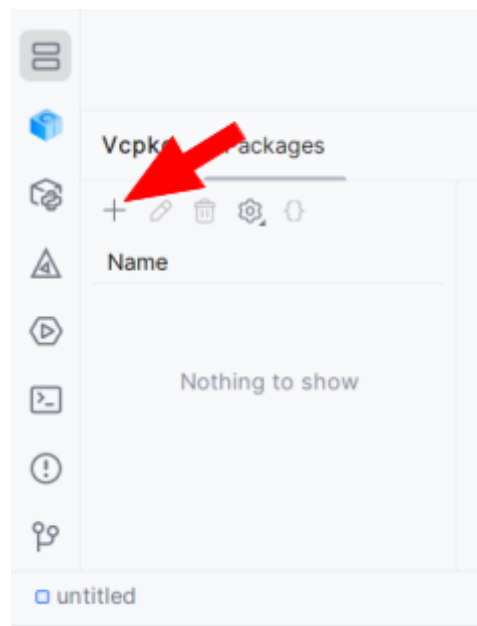


Figura A1.3

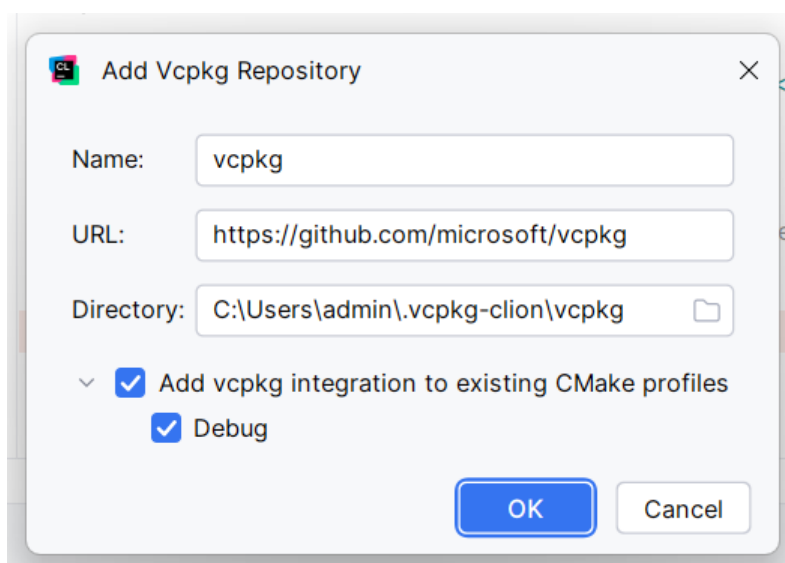


Figura A1.4

Como puedes ver en la Figura A1.4, puedes configurar tanto el nombre de la instancia de vcpkg (puedes querer tener una instancia para cada asignatura, por ejemplo) como la URL de descarga (no debes cambiarla) y el directorio de instalación de la instancia de vcpkg. Es en esta carpeta donde se descargarán los paquetes de las bibliotecas, por lo que debes asegurarte de que sea una ubicación de tu equipo con suficiente espacio.

³ Este sistema de instancias permite que podamos tener diferentes versiones de las bibliotecas descargadas en nuestro ordenador. Según la versión que queramos utilizar, agregaremos una u otra instancia de vcpkg al proyecto

Una vez hagas clic en OK, CLion se descargará la base del sistema vcpkg, y lo instalará en la carpeta que hayas configurado. Esto tardará varios minutos, dependiendo de la velocidad de tu conexión a Internet y la capacidad de procesamiento de tu ordenador.

Cuando termine la instalación, aparecerá la nueva instancia de vcpkg en el panel izquierdo, y podrás hacer clic en ella para utilizarla (Figura A1.5). Además, puedes hacer clic en el botón de la consola ("Console: vcpkg") si tienes curiosidad por ver los mensajes generados durante la descarga e instalación.

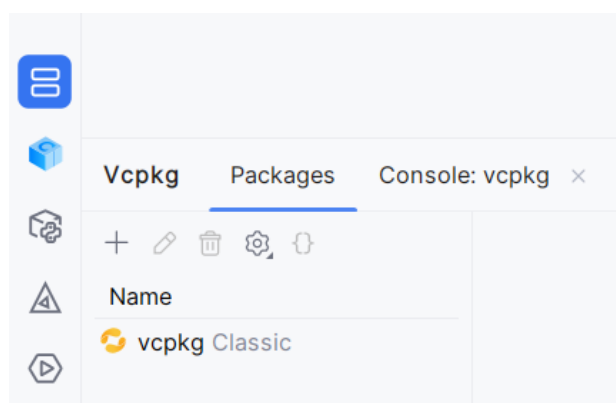


Figura A1.5

Tercer paso: añadir bibliotecas al proyecto

Para añadir a tu proyecto cualquier biblioteca a través de vcpkg, lo que tienes que hacer es:

1. Seleccionar la instancia de vcpkg vinculada a tu proyecto. Esta instancia aparecerá a color, mientras que si tienes otras instancias de vcpkg en tu equipo, aparecerán en gris pálido. Más adelante explicamos cómo cambiar la instancia de vcpkg vinculada a tu proyecto, por si te hiciera falta.
2. Buscar la biblioteca que necesitas. En el panel central del plugin de vcpkg tienes el cuadro de búsqueda. Puedes escribir el nombre de la biblioteca que necesitas, y si está disponible, aparecerá debajo (Figura A1.6).

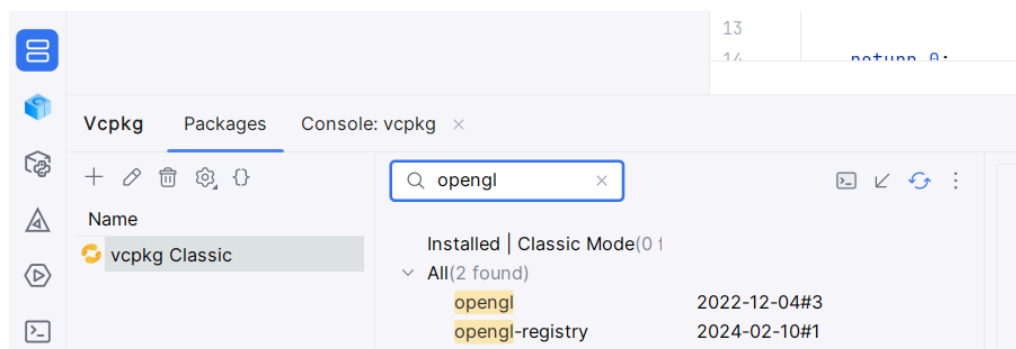


Figura A1.6

3. Haz clic en la biblioteca que quieres instalar, y en la parte derecha (Figura A1.7) aparecerá información detallada sobre la biblioteca, las versiones disponibles y dependencias con otras

bibliotecas. Además, aparecerá un botón de “Instalar” y un desplegable para elegir la configuración que quieres instalar (lo más habitual es que dejes la opción “dejar que vcpkg decida”).

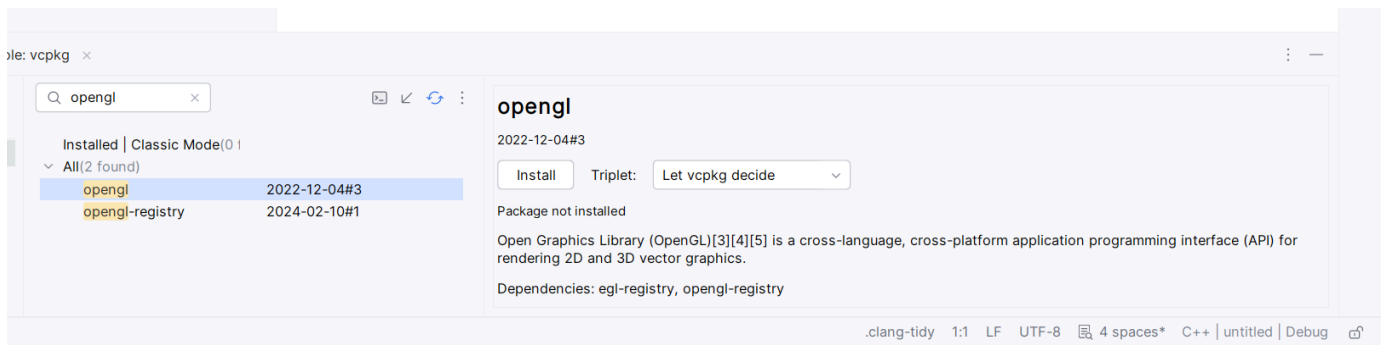


Figura A1.7

- Haz clic en el botón de “Instalar”, y deja a vcpkg hacer su trabajo. Si quieres, puedes abrir la consola para ver los mensajes que se van generando (Figura A1.8). Como puedes ver en la figura, los mensajes de cada proceso están contraídos, y haciendo clic en ellos puedes desplegarlos y leerlos.



Figura A1.8

- Al terminar la instalación, tendremos que añadir a nuestro archivo CMakeLists.txt las líneas necesarias para incluir la biblioteca que acabamos de instalar en el proceso de enlazado. Algunas veces, cuando termina la instalación aparece una ventana flotante en CLion con un enlace para mostrar las líneas que tienes que añadir (Figuras A1.9 y A1.10). Si esta ventana no aparece, o si la cierras por error, puedes consultar la consola de vcpkg, donde suele aparecer el mensaje con esas mismas líneas (Figura A1.11). Ten en cuenta que es posible que necesites adaptar las líneas a tu proyecto concreto, puesto que lo que se te muestra son ejemplos genéricos.

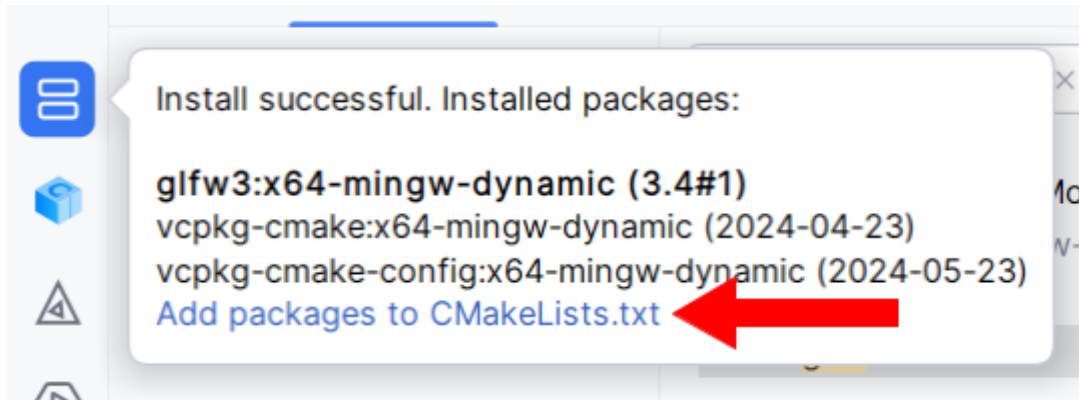


Figura A1.9

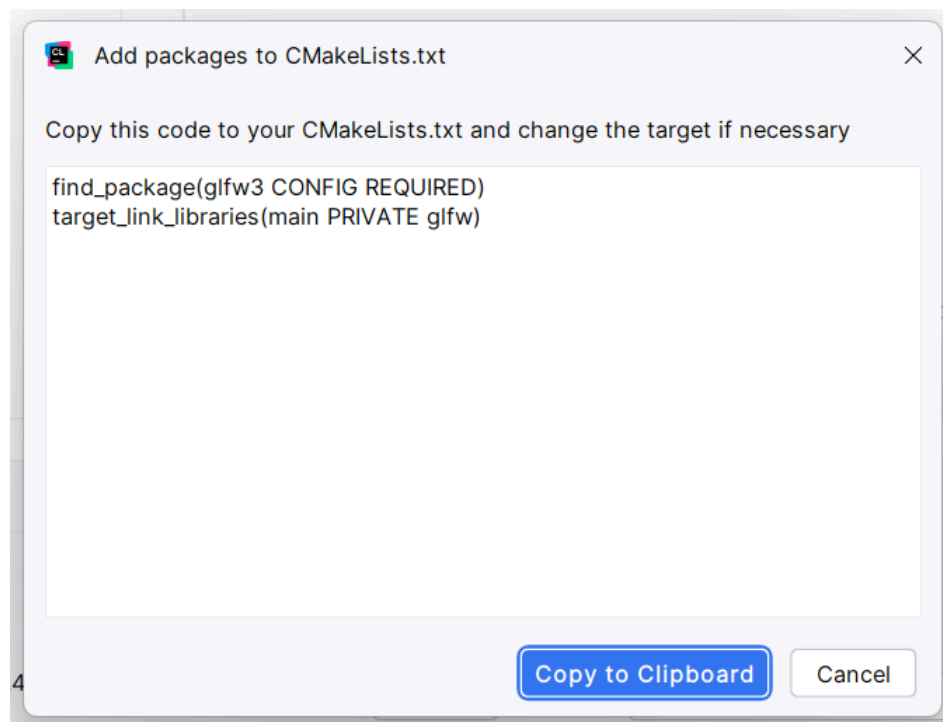


Figura A1.10

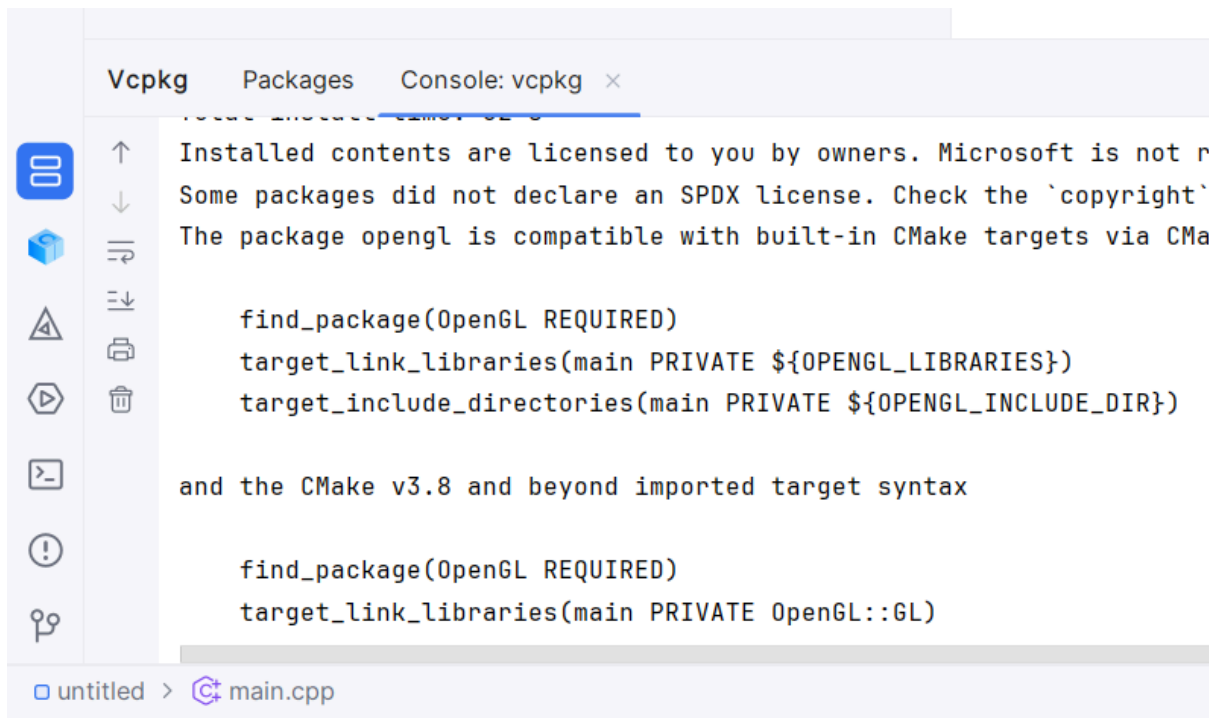


Figura A1.11

Presta atención a cómo hemos añadido las bibliotecas al proyecto (líneas 8, 9 y 10 de la Figura A1.12): las instrucciones se añaden después de la instrucción “add_executable”, y las instrucciones “find_package” se quedan tal cual se nos ofrecen, mientras que la instrucción “target_link_libraries” puede integrar en una sola todas las bibliotecas (en esta es imprescindible que el nombre del proyecto sea o bien el nombre de tu proyecto -“untitled” en la figura-, o bien la constante PROJECT_NAME).

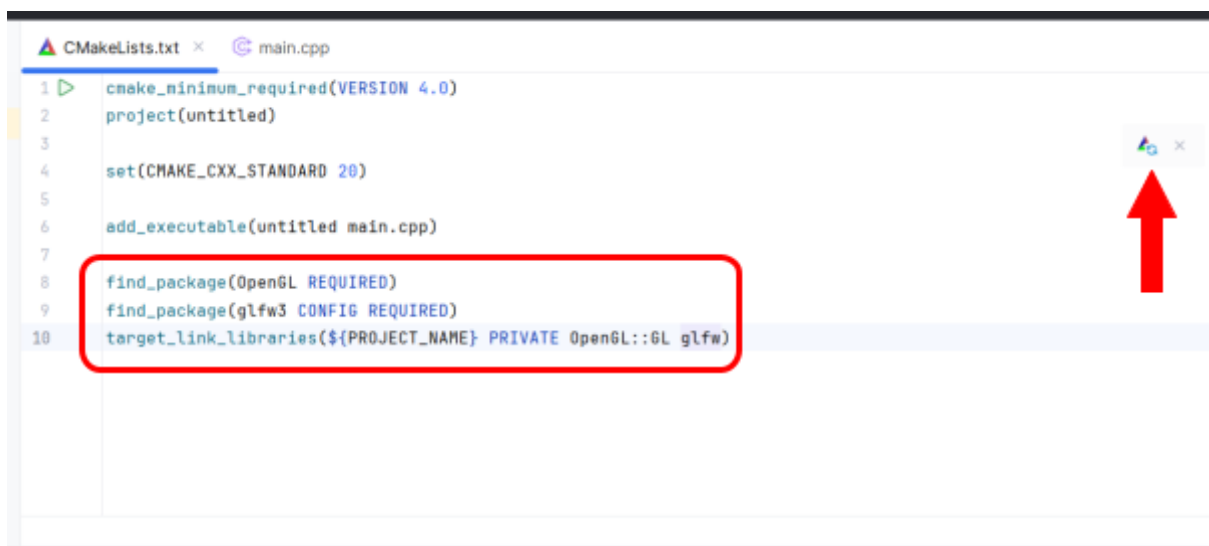


Figura A1.12

- Una vez añadida la configuración al archivo CMakeLists.txt, es necesario recargar la configuración del proyecto. Esto se puede hacer haciendo clic en el botón de “Recargar”

dentro del panel de CMake, o en el botón que aparece en el editor del CMakeLists.txt (Figura A1.12).

Cuarto paso: solución de problemas

Puede ser que al recargar el proyecto CMake nos aparezca un mensaje de error como el de la Figura A1.13. Este mensaje aparece porque CMake no sabe aún qué configuración de las bibliotecas tiene que utilizar. Vamos a ver cómo indicarla.

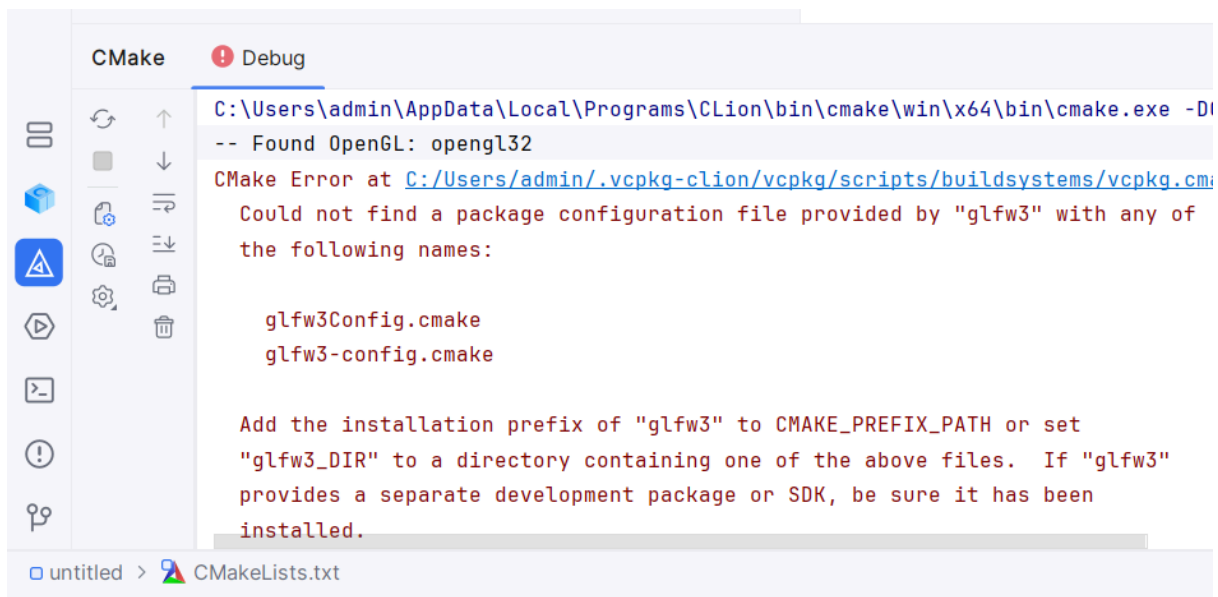


Figura A1.13

En la Figura A1.14 podemos ver que en el panel del plugin de vcpkg se muestra para cada biblioteca que tenemos instalada la “tripleta” (esto es: arquitectura-compilador-tipoDeEnlazado) que describe sus características. En la figura, la biblioteca está instalada con “x64-mingw-dynamic”, es decir: configuración de 64 bits, para su uso con el compilador MinGW (el que usa CLion) y enlazado dinámico. Tenemos que indicar a CMake que utilice esta tripleta a la hora de buscar las bibliotecas.

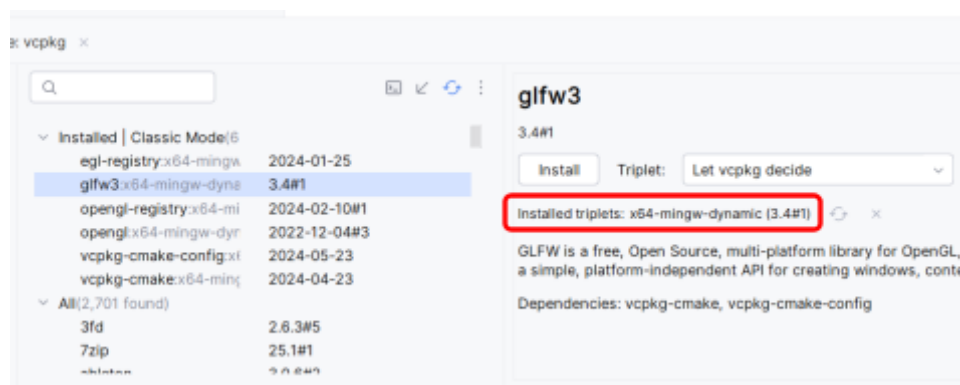


Figura A1.14

Para hacer esto, abrimos la configuración de CMake del proyecto, haciendo clic en el botón de configuración dentro del panel de CMake (Figura A1.15), y seleccionamos la opción “CMake settings”. Al hacer esto, se nos abrirá una nueva ventana (Figura A1.16). Asegúrate de que se ve bien todo el ancho de la ventana.

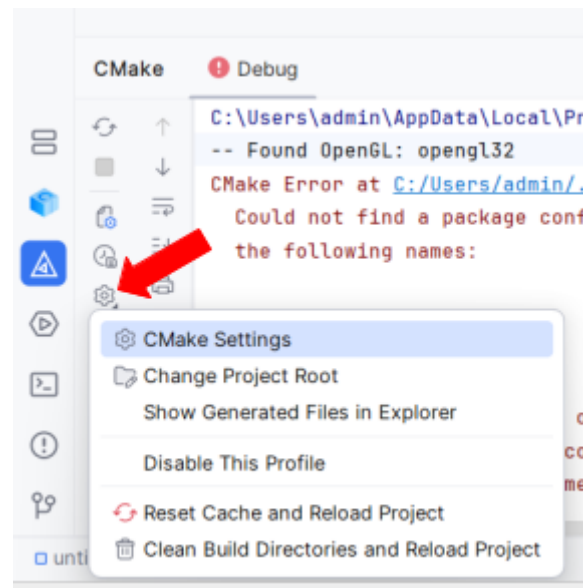


Figura A1.15

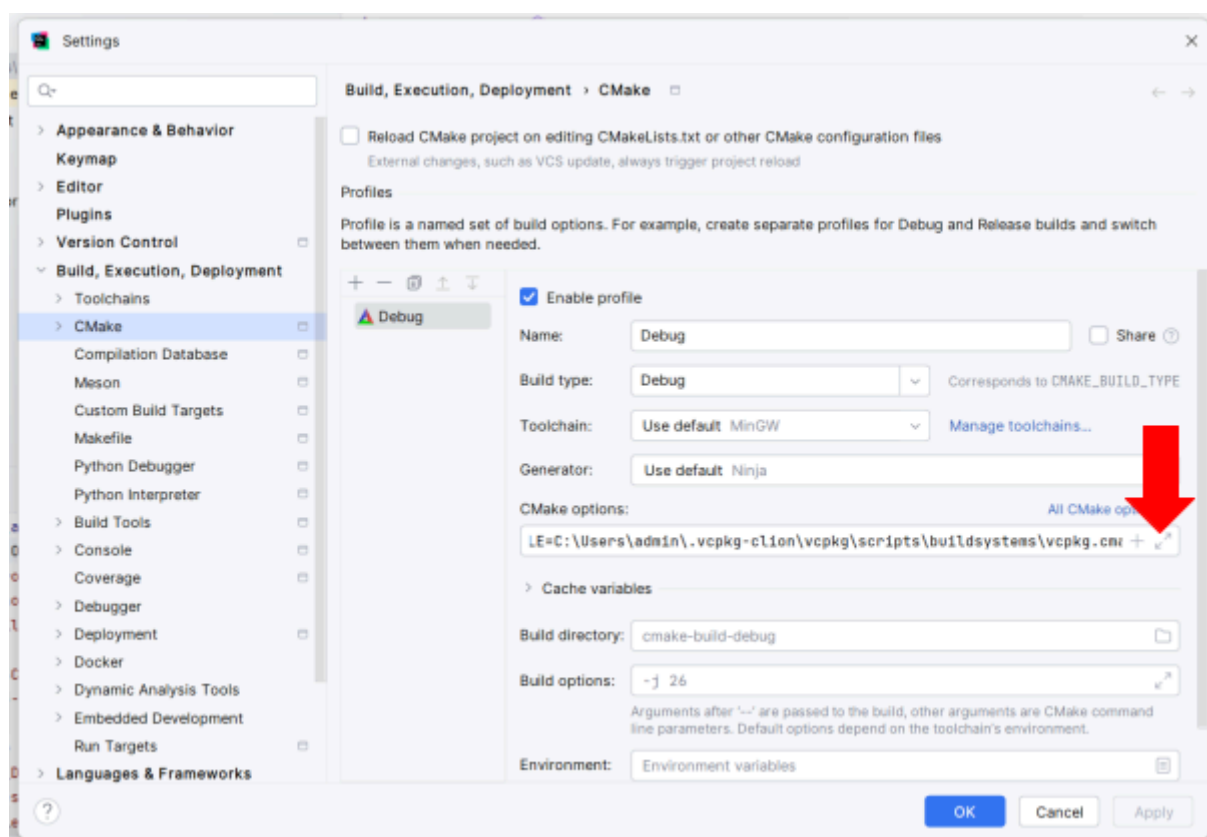


Figura A1.16

Buscamos la línea “CMake options”, y hacemos clic en el icono que despliega el cuadro, para que podamos editarlas con más facilidad (Figura A1.17). Una vez hecho esto, añadiremos la opción “-DVCPKG_TARGET_TRIPLET=x64-mingw-dynamic” (Figura A1.18). Recuerda que “x64-mingw-dynamic” es el valor de la tripleta que consultamos antes (Figura A1.14).

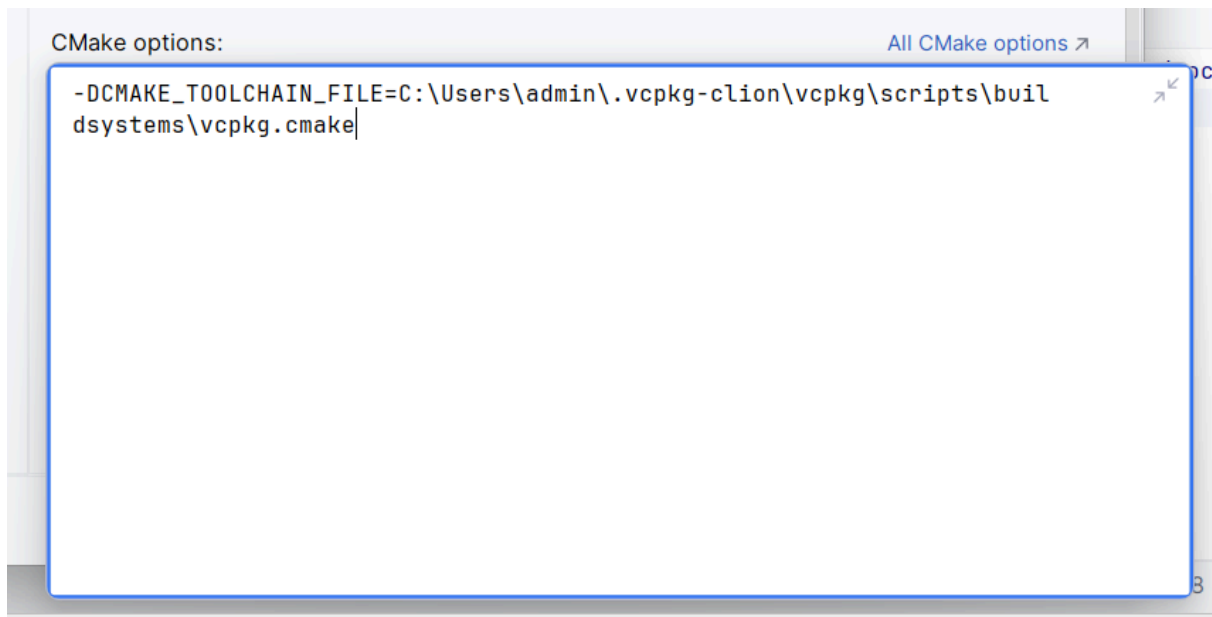


Figura A1.17

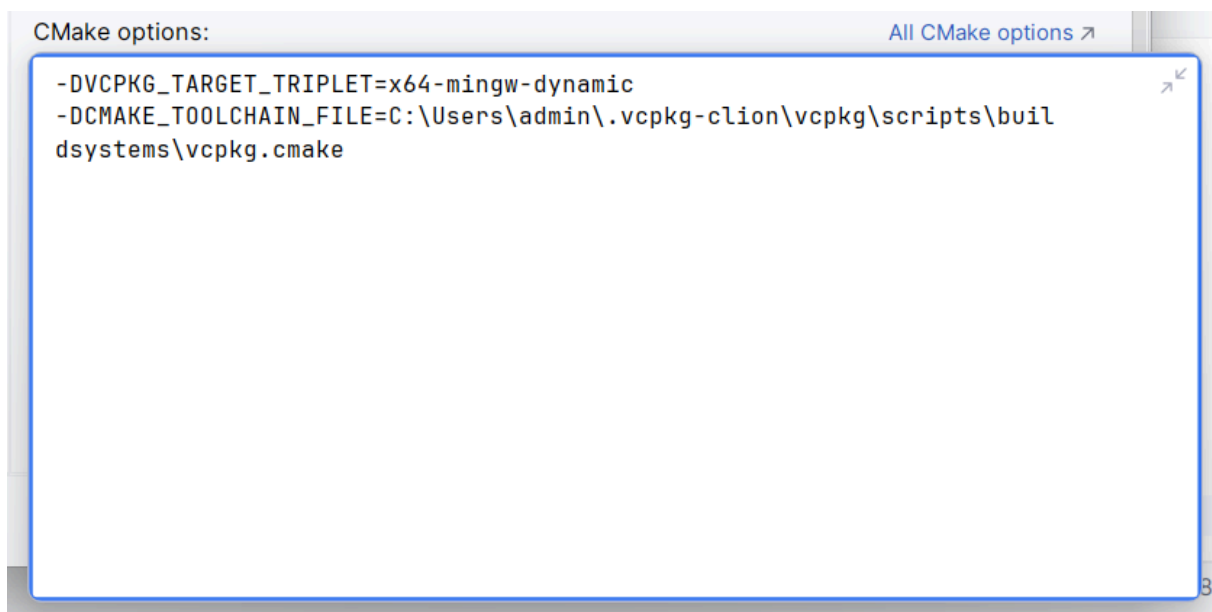


Figura A1.18

Ya podemos volver a plegar las opciones de CMake, y darle al botón de “OK”. El proyecto CMake se volverá a cargar automáticamente (y si no lo hiciera, lo hacemos manualmente), y ya debería funcionar todo correctamente.

Cambiar o añadir una instancia de vcpkg

Tanto si queremos cambiar la instancia de vcpkg que utilizamos en el proyecto, o si queremos añadir al proyecto una instancia vcpkg ya existente, lo único que tenemos que hacer es abrir el plugin de vcpkg (Figura A1.1), seleccionar la instancia de vcpkg que queremos utilizar, y darle al botón de “Editar” (Figura A1.19).

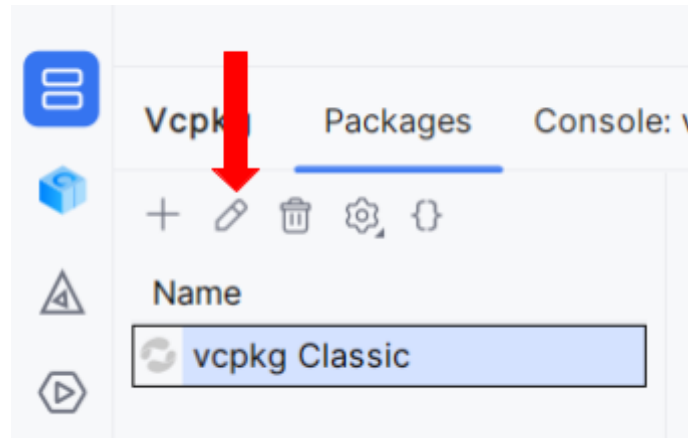


Figura A1.19

En la nueva ventana que aparece (Figura A1.20), marcaremos las opciones “Add vcpkg integration to existing CMake profiles” y “Debug”, y haremos clic en “OK”.

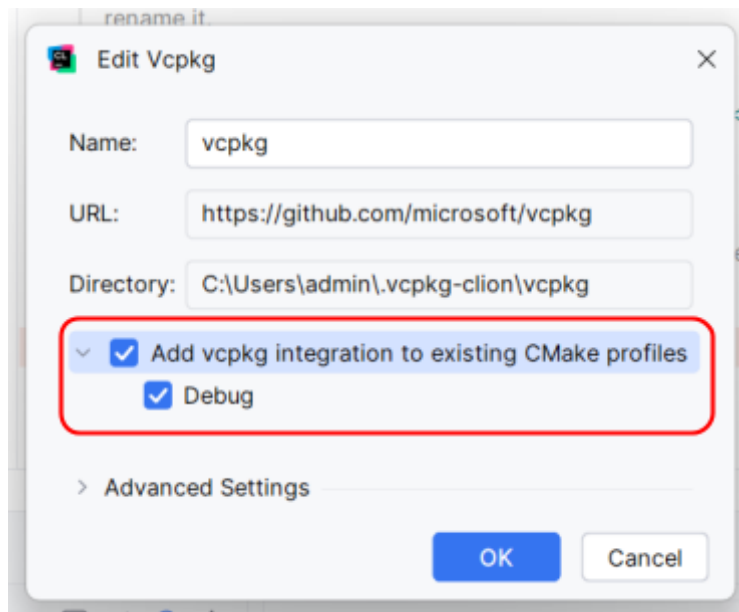


Figura A1.20

Ya podemos descargar e instalar bibliotecas en nuestro proyecto siguiendo los pasos descritos anteriormente.